Machine learning
- Binary linear classification

CdL Informatica e Innovazione Digitale

Valerio Freschi

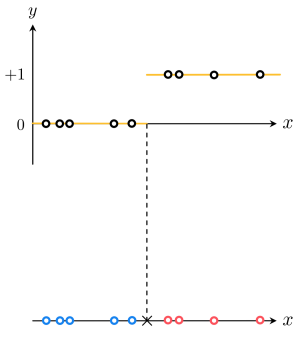
Università di Urbino - DiSPeA
valerio.freschi@uniurb.it

Linear two class classification

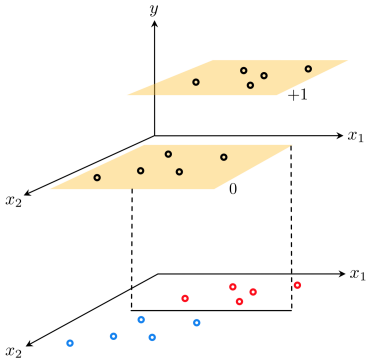
- Classification aims at predicting a discrete-value target
- If the set of possibile values is binary, we have a binary (or two class) classification problem
- Examples:
  - automatic medical diagnosis: predict if a patient has a disease or not, given specific symptoms
  - classify emails as spam/non-spam
  - predict if financial transactions are fraudulent
  - text based sentiment analysis (positive/negative opinion)
  - face detection: face/nonface objects

Linear two class classification

- Single input (left) and two input (right) example. Top: a regression view. Bottom: a perceptron view.



1d input: decision boundary is a point

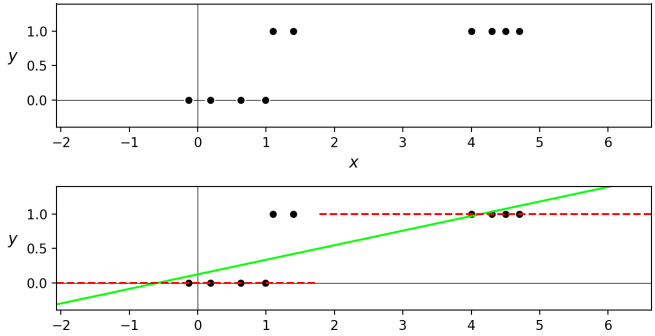


2d input: decision boundary is a line

Linear two class classification

- Green line is a linear regression fit.
- Poor data representation because output is discrete (2 values)
- Even if we use a discrete step function s.t.:
  - label $+1$ if *threshold* > 0.5
  - label 0 if *threshold* < 0.5

  is still problematic.

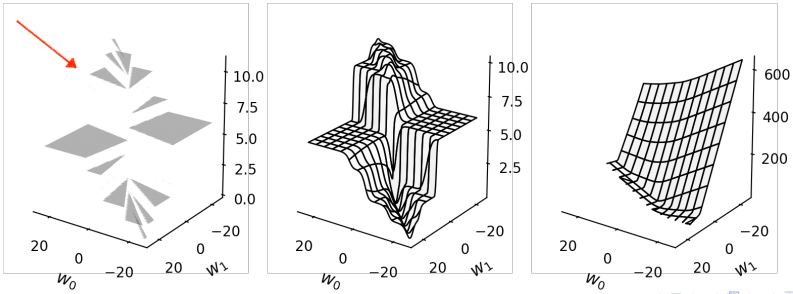


Linear two class classification

- Linear model of N-dimensional input: $\mathring{\boldsymbol{x}}^T \boldsymbol{w} = w_0 + x_1 w_1 + x_2 w_2 + \cdots + x_N w_N$
- What happens if we tune parameters after mapping through a *step* function?
  - $step(t) = 1 \quad \text{if } t > 0$
  - $step(t) = 0 \quad \text{if } t < 0$
- step $\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right)$
- Linear decision boundary between lower and upper steps, all points $\mathring{\boldsymbol{x}}$ where $\mathring{\boldsymbol{x}}^T \boldsymbol{w} = 0$
- To tune weight vector $\boldsymbol{w}$ we can build a Least Squares cost function $g(\mathrm{w})$

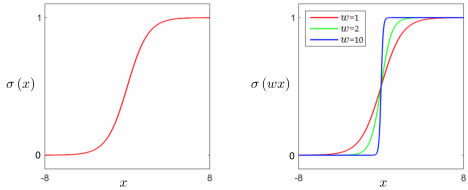
Linear two class classification

- Ideally, we want $(\boldsymbol{x}_p, y_p)$ to lie on the correct side of the decision boundary (i.e. output y_p on the proper step)
- step $\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right), \quad p = 1, ..., P$
- $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \left(\text{step}\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right) - y_p\right)^2$
- Very difficult to optimize this Least Squares cost function: completely flat locally (see example below)



Logistic regression

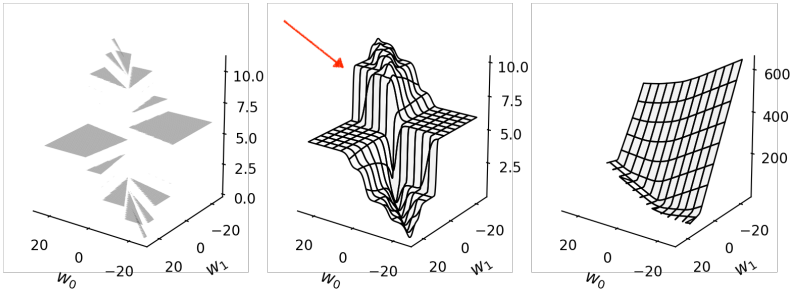
- The logistic sigmoid function
  - $\sigma(x) = \frac{1}{1+e^{-x}}$



- Logistic regression using the Least Squares cost
  - $\sigma\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right) \approx y_p \qquad p = 1, .., P$
  - Leading to the corresponding Least Squares cost function: $g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P}\left(\sigma\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right) - y_p\right)^2$
  - This logistic regression cost function is generally nonconvex
  - But it can be usually minimized by means of local optimization techniques

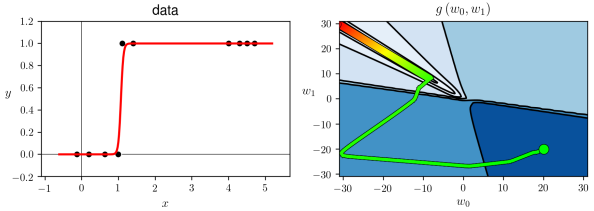
Linear two class classification

- Logistic regression using a Least Squares cost function: locally flat in many places, but it can be minimized by means of (normalized) gradient descent (middle).



Logistic regression

- Logistic regression using the Least Squares cost
  - Minimization of Least Squares cost with normalized gradient descent initialized at $w_0 = -w_1 = 20$.
  - Left: a sigmoidal fit obtained after minimization. Right: contour plot of cost function with normalized gradient descent path.

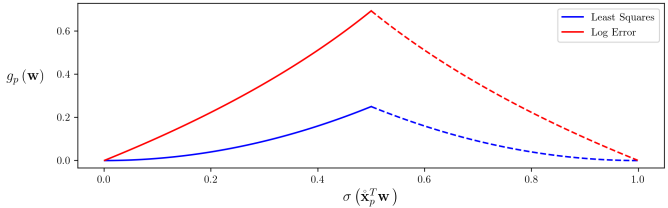


Logistic regression: Cross Entropy cost

- Logistic regression using the Cross Entropy cost
  - Introduce a point-wise cost (log error) $g_p(\boldsymbol{w}) = \begin{cases} -\log\left(\sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right) & \text{if } y_p = 1 \\ -\log\left(1 - \sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right) & \text{if } y_p = 0. \end{cases}$
  - and plug it into a cost function
- The logistic regression using the Cross Entropy cost function can be demonstrated to be convex
- Minimization can be achieved by means of local optimization techniques

Logistic regression

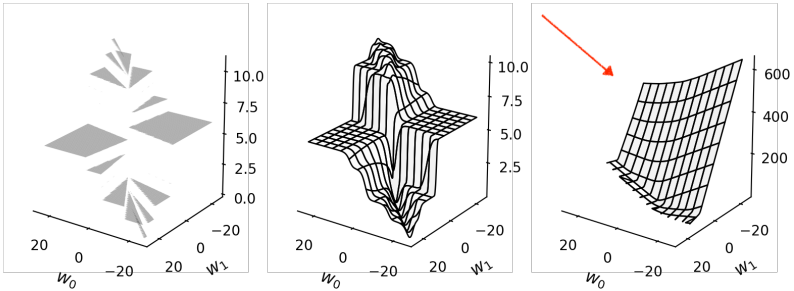
- Logistic regression: LS-CE cost functions comparison
  - Squared error (blue) vs log error (red).
  - $y_p = 0$, solid curves.
  - $y_p = 1$, dashed curves.
  - Notice how log error penalizes deviation from the true label to a greater extent w.r.t. squared error.



$$g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} g_p(\boldsymbol{w}) = -\frac{1}{P}\sum_{p=1}^{P} y_p \log\left(\sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right) + (1-y_p)\log\left(1-\sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right)$$

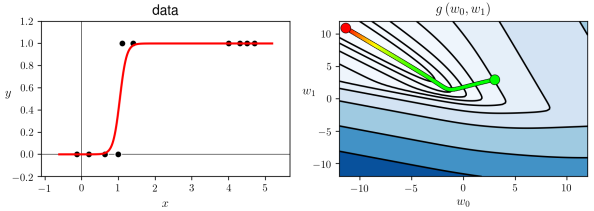
Linear two class classification

- Building a Cross Entropy cost function $g(\mathrm{w})$ results to be convex and can therefore be suitably minimized (right).
- Minimization can be achieved by means of local optimization schemes (e.g. standard gradient descent, Newton's method).



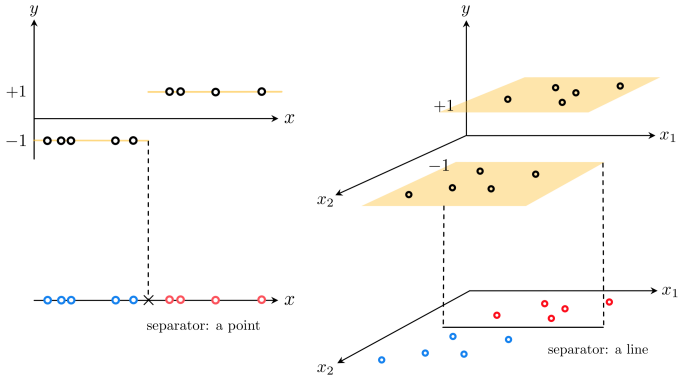
Logistic regression: Cross Entropy cost

- Logistic regression using the Cross Entropy cost
  - Minimization of Cross Entropy cost with standard gradient descent initialized at $w_0 = 3$ and $w_1 = 3$.
  - Left: a sigmoidal fit obtained after minimization. Right: contour plot of cost function with gradient descent path.



Linear two class classification

- Single input (left) and two input (right) example. Top: a regression view. Bottom: a perceptron view.



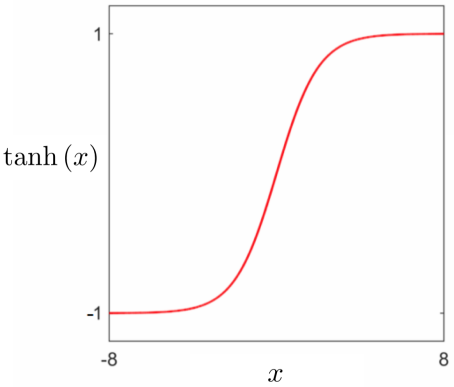
- Labels can be arbitrary
- What happens if $y_p \in \{-1, +1\}$ (and not $y_p \in \{0, +1\}$)?

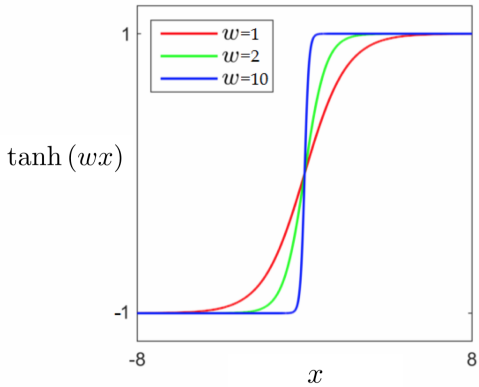
Logistic regression: Softmax cost

- A different algebraic formulation (*Softmax*) but equivalent to Cross Entropy cost.
- What happens if we tune parameters after mapping through a particular *step* function, called *sign* function?
  - $\text{sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$
  - $\text{sign}\left(\mathring{\boldsymbol{x}}^T \boldsymbol{w}\right)$
- Decision boundary: $\mathring{\boldsymbol{x}}$, where $\mathring{\boldsymbol{x}}^T \boldsymbol{w} = 0$
- If $\text{sign}\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = y_p$, the point is correctly classified.
- Building a Least Squares cost function directly using the sign function would lead to flat cost function (as before).
- Hence, replace the step function with a smooth approximation, namely *hyperbolic tangent function*

Logistic regression: Softmax cost

- $tanh(x) = 2\sigma(2x) - 1 = \frac{e^x - e^{-x}}{e^x + e^{-x}}$





Logistic regression: Softmax cost

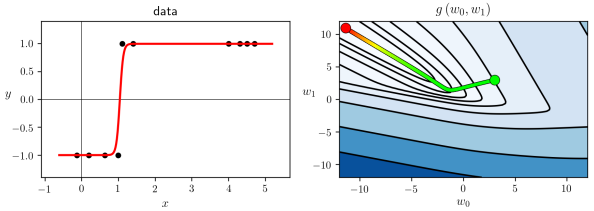
- The Least Squares cost function is (analogously to logistic regression with sigmoid) nonconvex.
- Introduce a point-wise log error cost as: $g_p(\boldsymbol{w}) = \begin{cases} -\log\left(\sigma\left(\mathring{\boldsymbol{x}}_p^T\boldsymbol{w}\right)\right) & \text{if } y_p = +1 \\ -\log\left(1-\sigma\left(\mathring{\boldsymbol{x}}_p^T\boldsymbol{w}\right)\right) & \text{if } y_p = -1. \end{cases}$
- and plug it into a cost function to build the *Softmax* cost function

$$g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} g_p(\boldsymbol{w})$$

- Like the Cross Entropy cost function, the Softmax Cost function can be proved to be always convex
- Apart the label change $y_p = -1 \leftrightarrow y_p = 0$ they are indeed completely equivalent

Logistic regression: Softmax cost

- Logistic regression using the Softmax cost
  - Minimization of Softmax cost with standard gradient descent initialized at $w_0 = 3$ and $w_1 = 3$.
  - Left: a sigmoidal (tanh) fit obtained after minimization. Right: contour plot of cost function with gradient descent path.



The Perceptron

- Logistic regression using the Softmax cost
  - Logistic regression learns a *nonlinear* regressor and its associated *linear* decision boundary.
  - The perceptron aims at learning the linear decision boundary directly (i.e. without learning any nonlinear regressor), leading back to a Softmax cost function.
  - Often: the same linear decision boundary.

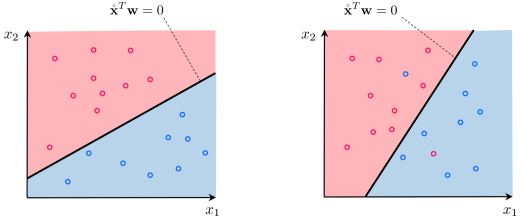


Figure: The decision boundary divides the input space in a half-space where $\mathring{\boldsymbol{x}}^T\boldsymbol{w} > 0$ (red) and a half-space where $\mathring{\boldsymbol{x}}^T\boldsymbol{w} < 0$ (blue).

The Perceptron

- $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} > 0 \quad$ if $y_p = +1$
  $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} < 0 \quad$ if $y_p = -1$.
- That can be consolidated into: $-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w} < 0$
- This ideal condition can be formalized as: $g_p(\boldsymbol{w}) = \max\left(0, -y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = 0$
- The functional form of this point-wise cost $\max(0, \cdot)$ is called *rectified linear unit*
- $g_p(\boldsymbol{w})$ is always nonnegative:
  - it returns 0 if $\boldsymbol{x}_p$ is classified correctly
  - it returns a positive value if $\boldsymbol{x}_p$ is classified incorrectly
- Hence, it is proper to average over the whole dataset:
  $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} g_p(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \max\left(0, -y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)$

The Perceptron

- $g(\boldsymbol{w})$ is known with different names: *ReLU cost*, *hinge cost*
- $g(\boldsymbol{w})$ is always convex, but with only one (discontinuous) derivative for each dimension
- $\Rightarrow$ only zero-order and first-order optimization methods can be used
- $g(\boldsymbol{w})$ always has trivial $\boldsymbol{w} = 0$ solution ($g(0) = 0$) $\Rightarrow$ take care to avoid it

The Perceptron

- To improve optimization issues we can replace the max function part of the Perceptron cost with a smooth (at least twice differentiable) alternative
- The *Softmax* function: $\text{soft}(s_0, s_1, ..., s_{C-1}) = \log\left(e^{s_0} + e^{s_1} + \cdots + e^{s_{C-1}}\right)$
  $\text{soft}(s_0, s_1, ..., s_{C-1}) \approx \max(s_0, s_1, ..., s_{C-1})$

The Perceptron

- To improve optimization issues we can replace the max function part of the Perceptron cost with a smooth (at least twice differentiable) alternative
- The *Softmax* function: $\text{soft}\left(s_0, s_1, ..., s_{C-1}\right) = \log\left(e^{s_0} + e^{s_1} + \cdots + e^{s_{C-1}}\right)$
  $\text{soft}\left(s_0, s_1, ..., s_{C-1}\right) \approx \max\left(s_0, s_1, ..., s_{C-1}\right)$
- $g_p\left(\boldsymbol{w}\right) = \text{soft}\left(0, -y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = \log\left(e^0 + e^{-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right) = \log\left(1 + e^{-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right)$

The Perceptron

- To improve optimization issues we can replace the max function part of the Perceptron cost with a smooth (at least twice differentiable) alternative
- The *Softmax* function: $\text{soft}\left(s_0, s_1, ..., s_{C-1}\right) = \log\left(e^{s_0} + e^{s_1} + \cdots + e^{s_{C-1}}\right)$
  $\text{soft}\left(s_0, s_1, ..., s_{C-1}\right) \approx \max\left(s_0, s_1, ..., s_{C-1}\right)$
- $g_p\left(\boldsymbol{w}\right) = \text{soft}\left(0, -y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = \log\left(e^0 + e^{-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right) = \log\left(1 + e^{-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right)$
- $g\left(\boldsymbol{w}\right) = \frac{1}{P}\sum_{p=1}^{P} g_p\left(\boldsymbol{w}\right) = \frac{1}{P}\sum_{p=1}^{P} \log\left(1 + e^{-y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right)$

The Perceptron

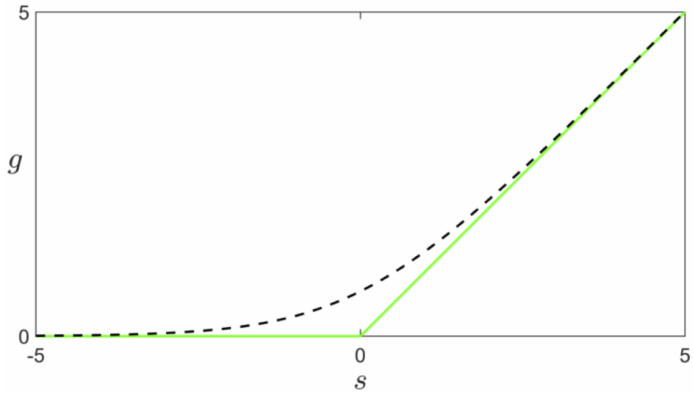


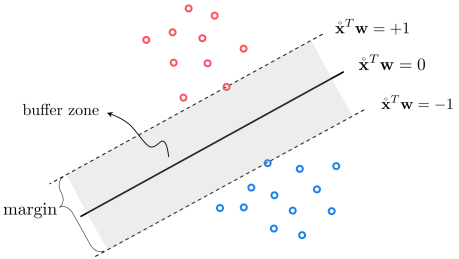
Figure: $g(s) = max(0, s)$ (green) and its smooth Softmax approximation $g(s) = soft(0, s) = log(1 + e^s)$ (dashed black).

The Perceptron

- $g(\boldsymbol{w})$ is the Softmax cost previously derived for logistic regression approach on binary classification (from which the name)
- Like the Perceptron cost, the Softmax cost is convex
- It has ∞ derivatives (and Newton's method can be used to minimize it)
- No trivial 0 solution
- Little difference in practice between logistic regression and perceptron perspectives on classification (no difference when Softmax is applied to Perceptron)

Support Vector Machines

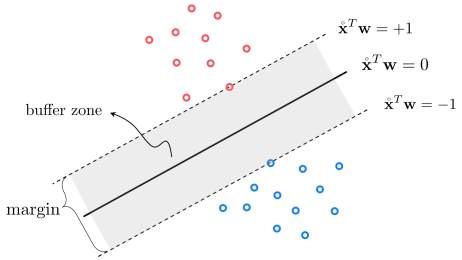
- SVM are a popular model for linear classification
- They can also be used (properly modified) for nonlinear classification
- Basic concept: separate example by wide margin boundary



- Suppose the training dataset $\{(\boldsymbol{x}_p, y_p)\}_{p=1}^{P}$ with labels $y_p \in \{-1, +1\}$ is perfectly linarly spearable
- Decision boundary: $\mathring{\boldsymbol{x}}^T \boldsymbol{w} = 0$

Support Vector Machines

- The Margin-Perceptron
- The separating hyperplane creates a buffer between the two classes



- All points in the $+1$ class lie on the positive side of $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} = +1$; All points in the $+1$ class lie on the positive side of $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} = -1$;
- $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} \geq 1 \quad$ if $y_p = +1$
  $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} \leq -1 \quad$ if $y_p = -1$

Support Vector Machines

- The two inequalities can be combined into one

Support Vector Machines

- The two inequalities can be combined into one
- $g_p(\boldsymbol{w}) = \max\left(0, 1 - y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = 0$

Support Vector Machines

- The two inequalities can be combined into one
- $g_p(\boldsymbol{w}) = \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p{}^T \boldsymbol{w}\right) = 0$
- $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p{}^T \boldsymbol{w}\right)$

Support Vector Machines

- The two inequalities can be combined into one
- $g_p(\boldsymbol{w}) = \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = 0$
- $g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)$
- Relation to the Softmax cost: What if we change the max operator with Softmax? (Remember what we did with the Perceptron)

Support Vector Machines

- The two inequalities can be combined into one
- $g_p(\boldsymbol{w}) = \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = 0$
- $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \max\left(0,\, 1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)$
- Relation to the Softmax cost: What if we change the max operator with Softmax? (Remember what we did with the Perceptron)
- $\text{soft}\left(0, 1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) = \log\left(1 + e^{1 - y_p\, \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}}\right)$
- Averaging over P leads to a cost function close to the Margin-Perceptron

Support Vector Machines

- Any value $\epsilon > 0$ could replace the 1 used: $g(\boldsymbol{w}) = \sum_{p=1}^{P} \max\left(0, \epsilon - y_p \mathring{\boldsymbol{x}}_p{}^T \boldsymbol{w}\right)$

Support Vector Machines

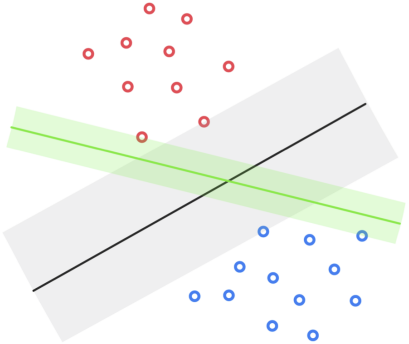
- Any value $\epsilon > 0$ could replace the 1 used: $g(\boldsymbol{w}) = \sum_{p=1}^{P} \max\left(0,\, \epsilon - y_p \mathring{\boldsymbol{x}}_p{}^T \boldsymbol{w}\right)$
- soft $\left(0, \epsilon - y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}\right) = \log\left(1 + e^{\epsilon - y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}}\right)$

Support Vector Machines

- Any value $\epsilon > 0$ could replace the 1 used: $g(\boldsymbol{w}) = \sum_{p=1}^{P} \max\left(0,\, \epsilon - y_p \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)$
- soft $\left(0, \epsilon - y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}\right) = \log\left(1 + e^{\epsilon - y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}}\right)$
- Hence, for small ϵ, $\log\left(1 + e^{\epsilon - y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}}\right) \approx \log\left(1 + e^{-y_p \mathring{\boldsymbol{x}}^T \boldsymbol{w}}\right)$
- The Softmax cost function can be seen as a smoothed version of the Margin-Perceptron cost

Support Vector Machines

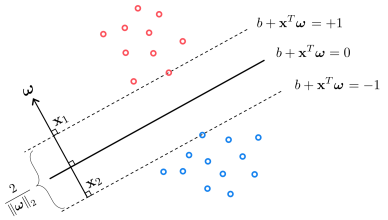
- Maximum margin decision boundaries
- For perfectly linearly separable data there are ∞ hyperplanes that perfectly divide data



- Which is the best one?
- Margin lengths: the larger the distance the better the separation and the possibility to avoid misclassification for future data points

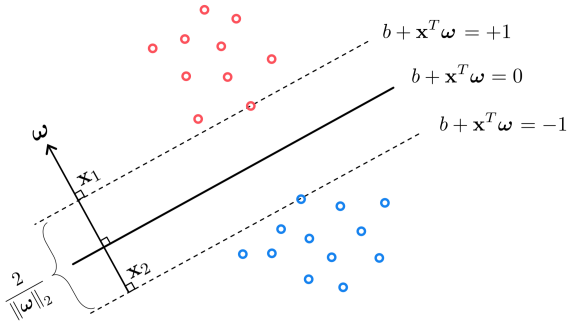
Support Vector Machines

- Maximum margin decision boundaries.
- Linear decision boundary: $\mathring{\boldsymbol{x}}^T \boldsymbol{w} = b + \boldsymbol{x}^T \boldsymbol{\omega} = 0$
- We aim at finding the parameters so that the region defined by $b + \boldsymbol{x}^T \boldsymbol{\omega} = \pm 1$ has the largest margin
- Given: $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ the points on the normal vector $\boldsymbol{\omega}$ that $\in$ positive and negative translated hyperplanes



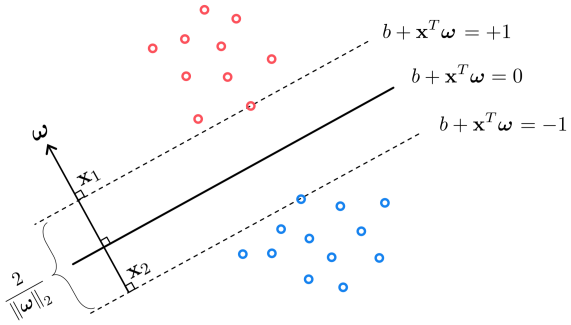
- The margin is represented by $\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2$

Support Vector Machines



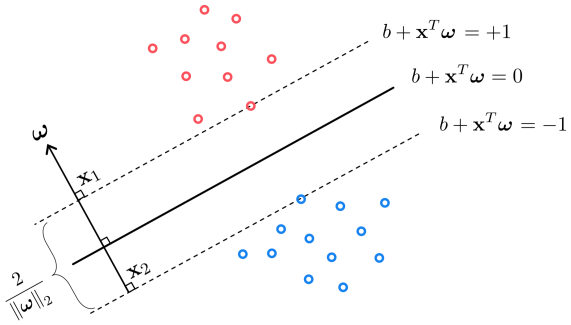
- $(\boldsymbol{x}_1 - \boldsymbol{x}_2)^T\boldsymbol{\omega} = 2$

Support Vector Machines



- $(\boldsymbol{x}_1 - \boldsymbol{x}_2)^T\boldsymbol{\omega} = 2$
- $2 = (\boldsymbol{x}_1 - \boldsymbol{x}_2)^T\boldsymbol{\omega} = \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2 \, \|\boldsymbol{\omega}\|_2$

Support Vector Machines



- $(\boldsymbol{x}_1 - \boldsymbol{x}_2)^T\boldsymbol{\omega} = 2$
- $2 = (\boldsymbol{x}_1 - \boldsymbol{x}_2)^T\boldsymbol{\omega} = \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2 \|\boldsymbol{\omega}\|_2$
- $\|\boldsymbol{x}_1 - \boldsymbol{x}_2\|_2 = \frac{2}{\|\boldsymbol{\omega}\|_2}$

Support Vector Machines

- Hard and soft margins
- To find a hyperplane with min length normal vector:
  - we need to minimize $\|\boldsymbol{\omega}\|_2^2$
  - Subject to the constraint that the hyperplane perfectly separates data
- This is the *hard-margin* SVM problem:

  $$\underset{b,\boldsymbol{\omega}}{\text{minimize}} \ \|\boldsymbol{\omega}\|_2^2$$

  $$\text{subject to} \ \max\left(0,\, 1 - y_p\left(b + \boldsymbol{x}_p^T\boldsymbol{\omega}\right)\right) = 0, \quad p = 1, ..., P.$$
- Constrained optimization algorithms need to be used
- Alternatively, we can relax constraints (by means of *regularization* and get a unconstrained formulation

Support Vector Machines

- Hard and soft margins
- To find a hyperplane with min length normal vector:
  - we need to minimize $\|\boldsymbol{\omega}\|_2^2$
  - Subject to the constraint that the hyperplane perfectly separates data
- This is the *hard-margin* SVM problem:

  $$\underset{b,\boldsymbol{\omega}}{\text{minimize}} \ \|\boldsymbol{\omega}\|_2^2$$

  $$\text{subject to} \ \max\left(0,\, 1 - y_p\left(b + \boldsymbol{x}_p^T\boldsymbol{\omega}\right)\right) = 0, \quad p = 1, ..., P.$$
- Constrained optimization algorithms need to be used
- Alternatively, we can relax constraints (by means of *regularization* and get a unconstrained formulation
- $g\left(b, \boldsymbol{\omega}\right) = \frac{1}{P}\sum_{p=1}^{P} \max\left(0,\, 1 - y_p\left(b + \boldsymbol{x}_p^T\boldsymbol{\omega}\right)\right) + \lambda\left\|\boldsymbol{\omega}\right\|_2^2$

Support Vector Machines

- Regularization approach: $g\left(b, \boldsymbol{\omega}\right) = \frac{1}{P} \sum_{p=1}^{P} \max\left(0,\, 1 - y_p \left(b + \boldsymbol{x}_p^T \boldsymbol{\omega}\right)\right) + \lambda \left\|\boldsymbol{\omega}\right\|_2^2$
- $\lambda \geq 0$: penalty/regularization parameter
- In practice a small value (if small we put pressure to satisfy constraints, if very small we tend to match the constrained formulation)
- This regularized version of the Margin-Perceptron cost function is known as *soft-margin* SVM cost function

Support Vector Machines

- In case of noisy data, constraints in hard-margin SVM could become impossible to satisfy, while soft-margin SVM cost function can be minimized (more used in practice)
- Also, smoothed version of soft-margin SVM cost function can be used:
  $g(b, \boldsymbol{\omega}) = \frac{1}{P} \sum_{p=1}^{P} \log\left(1 + e^{-y_p\left(b + \boldsymbol{x}_p^T \boldsymbol{\omega}\right)}\right) + \lambda \|\boldsymbol{\omega}\|_2^2$
- It's also a regularized Softmax cost!
- $\Rightarrow$ Logistic regression, Perceptron, and SVMs are intertwined and lead to similar results on not linearly separable datasets

Support Vector Machines

- The SVM decision boundary example

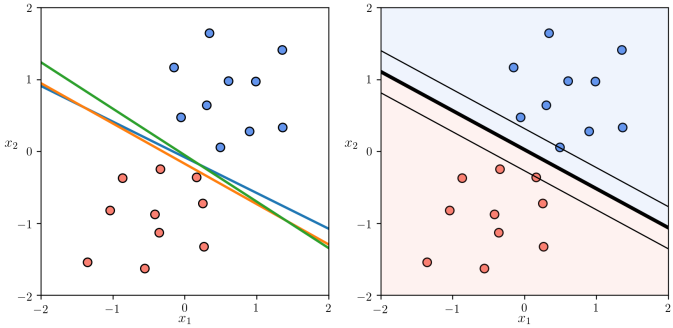


Figure: Boundaries learned by minimizing the Margin-Perceptron cost (left), and by minimzing the soft-margin SVM cost (with regularization parameter $\lambda = 10^{-3}$).

- The translates of decision boundary pass through points from both classes (equidistant from the SVM decision boundary) called *support vectors*

Categorical labels: the Cross Entropy cost

- For logistic regression that $y_p \in \{0, 1\}$ or $y_p \in \{-1, +1\}$ result into the same binary classification when Cross Entropy /Softmax cost functions are minimized
- We form log error point-wise costs, average over P, and derive convex cost functions
- Can we derive the same result for categorical labels?
- One-hot encoded categorical labels:
  - Wlog: suppose a two-class dataset $\{(\boldsymbol{x}_p, y_p)\}_{p=1}^{P}$
  - Transform label values $y_p \in \{0, 1\}$ into one-hot encoded vectors:

    $y_p = 0 \longleftarrow \boldsymbol{y}_p = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad y_p = 1 \longleftarrow \boldsymbol{y}_p = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
  - No longer ordered numerical values, but always want to *regress* input to output

Categorical labels: the Cross Entropy cost

- Classification task is viewed as a multi-output regression
- We map N-dimensional input to two-dimensional output using a nonlinear function of the linear function $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}$
- Nonlinearity is needed because we want two outputs
- $\boldsymbol{\sigma}_p = \begin{bmatrix} \sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) \\ 1 - \sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right) \end{bmatrix}$
- Thanks to this nonlinear transformation a proper tuning of weights $\boldsymbol{w}$ will impose $\boldsymbol{\sigma}_p \approx \boldsymbol{y}_p$

Categorical labels: the Cross Entropy cost

- Similarly to numerical labels we can introduce e point-wise cost
- Given the binary form of labels is better using a log error:
  $g_p(\boldsymbol{w}) = -\boldsymbol{y}_p^T \log \boldsymbol{\sigma}_p = -y_{p,1} \log\left(\sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right) - y_{p,2} \log\left(1 - \sigma\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}\right)\right)$
- Averaged over the P point-wise costs gives the *Categorical Cross Entropy* cost function
- This ends up into the log error found for numerical labels

Classification quality metrics

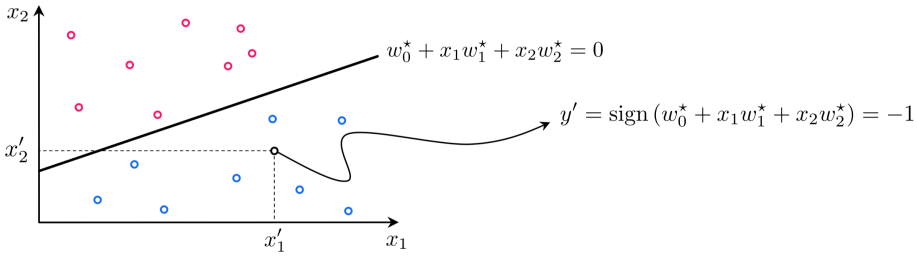
- How can we make predictions with a trained model?
- $\text{model}\,(\boldsymbol{x}, \boldsymbol{w}^\star) = \mathring{\boldsymbol{x}}^T \boldsymbol{w}^\star = w_0^\star + x_1 w_1^\star + x_2 w_2^\star + \cdots + x_N w_N^\star$

Classification quality metrics

- How can we make predictions with a trained model?
- $\text{model}(\boldsymbol{x}, \boldsymbol{w}^\star) = \mathring{\boldsymbol{x}}^T \boldsymbol{w}^\star = w_0^\star + x_1 w_1^\star + x_2 w_2^\star + \cdots + x_N w_N^\star$
- $\text{model}(\boldsymbol{x}, \boldsymbol{w}^\star) = 0$

Classification quality metrics

- How can we make predictions with a trained model?
- $\text{model}(\boldsymbol{x}, \boldsymbol{w}^\star) = \mathring{\boldsymbol{x}}^T \boldsymbol{w}^\star = w_0^\star + x_1 w_1^\star + x_2 w_2^\star + \cdots + x_N w_N^\star$
- $\text{model}(\boldsymbol{x}, \boldsymbol{w}^\star) = 0$
- $\text{sign}\,(\text{model}(\boldsymbol{x}, \boldsymbol{w}^\star)) = y$



Classification quality metrics

- Confidence scoring
- The point's distance form the decision boundary
- This intuitive concept can be measured by plugging it into a sigmoid function
- $d = \frac{b^\star + \boldsymbol{x}_p^T \boldsymbol{\omega}^\star}{\|\boldsymbol{\omega}^\star\|_2}$
- (bias): $b^\star = w_0^\star$ (feature-touching weights): $\boldsymbol{\omega}^\star = \begin{bmatrix} w_1^\star \\ w_2^\star \\ \vdots \\ w_N^\star \end{bmatrix}$
- confidence in the predicted label of a point $\boldsymbol{x}$ $= \sigma(d)$
- Since $\sigma : (-\infty, +\infty) \to [0, 1]$ it can be interpreted as a probability

Classification quality metrics

- We can evaluate the quality of a trained model by means of given metrics, e.g. the *accuracy*

Classification quality metrics

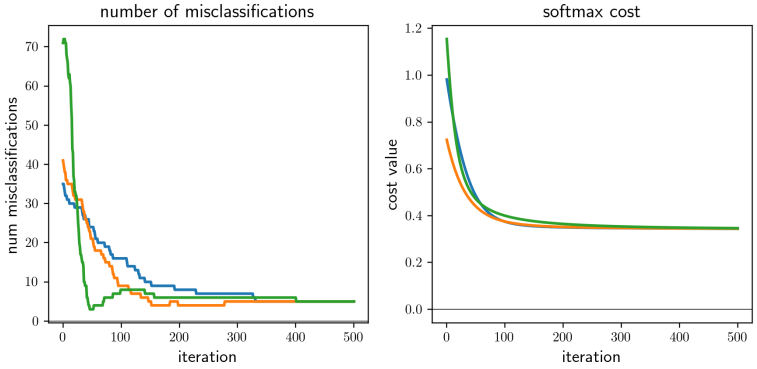
- We can evaluate the quality of a trained model by means of given metrics, e.g. the *accuracy*
- $\hat{y}_p = \text{sign}\left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}^\star\right)\right)$

Classification quality metrics

- We can evaluate the quality of a trained model by means of given metrics, e.g. the *accuracy*
- $\hat{y}_p = \text{sign}\left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}^\star\right)\right)$
- number of misclassifications $= \sum_{p=1}^{P} \mathcal{I}\left(\hat{y}_p, y_p\right)$

Classification quality metrics

- We can evaluate the quality of a trained model by means of given metrics, e.g. the *accuracy*
- $\hat{y}_p = \text{sign}\left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}^\star\right)\right)$
- number of misclassifications $= \sum_{p=1}^{P} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A} = 1 - \frac{1}{P} \sum_{p=1}^{P} \mathcal{I}\left(\hat{y}_p, y_p\right)$



Classification quality metrics

- Accuracy can be misleading in the case of highly imbalanced classes
- Suppose one class accounts for 95% of points
- A naive classifier that assigns to every training point the majority class achieves $\mathcal{A} = 95\%$
- However it completely misclassifies the other class
- This could be an issue for instance in health-related (e.g. healthy vs rare disease suffering people) or financial fraud detection (e.g. valid vs fraudulent transactions)
- Standard accuracy equally weights misclassification from both classes
- *Balanced accuracy* is to compute accuracy on each class and take the average

Classification quality metrics

- Let Ω_{+1} and Ω_{-1} be the indices of points will labels $y_p = +1$ and $y_p = -1$

Classification quality metrics

- Let Ω_{+1} and Ω_{-1} be the indices of points will labels $y_p = +1$ and $y_p = -1$
- number of misclassifications on $+1$ class $= \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  number of misclassifications on -1 class $= \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$

Classification quality metrics

- Let Ω_{+1} and Ω_{-1} be the indices of points will labels $y_p = +1$ and $y_p = -1$
- number of misclassifications on $+1$ class $= \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  number of misclassifications on -1 class $= \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A}_{+1} = 1 - \frac{1}{|\Omega_{+1}|} \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  $\mathcal{A}_{-1} = 1 - \frac{1}{|\Omega_{-1}|} \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$

Classification quality metrics

- Let Ω_{+1} and Ω_{-1} be the indices of points will labels $y_p = +1$ and $y_p = -1$
- number of misclassifications on $+1$ class $= \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  number of misclassifications on -1 class $= \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A}_{+1} = 1 - \frac{1}{|\Omega_{+1}|} \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  $\mathcal{A}_{-1} = 1 - \frac{1}{|\Omega_{-1}|} \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A}_{\text{balanced}} = \frac{\mathcal{A}_{+1} + \mathcal{A}_{-1}}{2}$

Classification quality metrics

- Let Ω_{+1} and Ω_{-1} be the indices of points will labels $y_p = +1$ and $y_p = -1$
- number of misclassifications on $+1$ class $= \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  number of misclassifications on -1 class $= \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A}_{+1} = 1 - \frac{1}{|\Omega_{+1}|} \sum_{p \in \Omega_{+1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
  $\mathcal{A}_{-1} = 1 - \frac{1}{|\Omega_{-1}|} \sum_{p \in \Omega_{-1}} \mathcal{I}\left(\hat{y}_p, y_p\right)$
- $\mathcal{A}_{\text{balanced}} = \frac{\mathcal{A}_{+1} + \mathcal{A}_{-1}}{2}$
- $0 \leq \mathcal{A}_{\text{balanced}} \leq 1$ ($= 0.5$ in the $5\% - 95\%$ example)

Classification quality metrics

- From the *confusion matrix* additional metrics can be derived

$$
\begin{array}{cc} & \text{predicted label} \\ & \begin{array}{cc} +1 & -1 \end{array} \\ \text{actual label} \begin{array}{c} +1 \\ -1 \end{array} & \begin{pmatrix} A & B \\ C & D \end{pmatrix} \end{array}
$$

Classification quality metrics

- From the *confusion matrix* additional metrics can be derived

$$\begin{array}{cc} & \text{predicted label} \\ & \begin{array}{cc} +1 & -1 \end{array} \\ \text{actual label} \begin{array}{c} +1 \\ -1 \end{array} & \begin{pmatrix} A & B \\ C & D \end{pmatrix} \end{array}$$

- $\mathcal{A} = \frac{A+D}{A+B+C+D}$

Classification quality metrics

- From the *confusion matrix* additional metrics can be derived

$$\begin{array}{cc} & \text{predicted label} \\ & \begin{array}{cc} +1 & -1 \end{array} \\ \text{actual label} \begin{array}{c} +1 \\ -1 \end{array} & \begin{pmatrix} A & B \\ C & D \end{pmatrix} \end{array}$$

- $\mathcal{A} = \frac{A+D}{A+B+C+D}$
- $\mathcal{A}_{+1} = \frac{A}{A+B}$ (*sensitivity*)
  $\mathcal{A}_{-1} = \frac{D}{C+D}$ (*specificity*)

Classification quality metrics

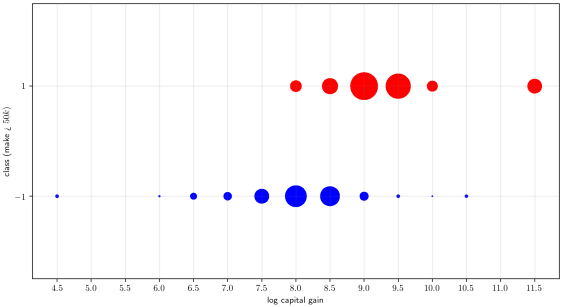
- From the *confusion matrix* additional metrics can be derived

$$\begin{array}{cc} & \text{predicted label} \\ & \begin{array}{cc} +1 & -1 \end{array} \\ \text{actual label} \begin{array}{c} +1 \\ -1 \end{array} & \begin{pmatrix} A & B \\ C & D \end{pmatrix} \end{array}$$

- $\mathcal{A} = \frac{A+D}{A+B+C+D}$
- $\mathcal{A}_{+1} = \frac{A}{A+B}$ (*sensitivity*)
  $\mathcal{A}_{-1} = \frac{D}{C+D}$ (*specificity*)
- $\mathcal{A}_{\text{balanced}} = \frac{1}{2}\frac{A}{A+B} + \frac{1}{2}\frac{D}{C+D}$

Weighted binary classification

- In principle individual points can be (de)emphasised by proper weighting (because cost functions are sums over individual points)
- Repeated points should be evaluated (as for regression)



- By introducing point-wise weights β_1, β_2, ..., β_P we may write the weighted Softmax cost
- $g(\boldsymbol{w}) = \frac{1}{\beta_1+\beta_2+...\beta_P} \sum_{p=1}^{P} \beta_p \log\left(1 + e^{-y_p \text{model}(x_p, \boldsymbol{w})}\right)$

Weighted binary classification

- Weights can be used to account for repeated points
- i.e. $\beta_p = 1$ for a unique point (x_p, y_p)
- and $\beta_p = R$ and $\beta_p = 0$ for others if the point is repeated R times
- Repeated points should be evaluated (as for regression)
- Weights are fixed $\Rightarrow$ weighted classification costs can be minimized by means of local optimization

Weighted binary classification

- Weights can be used to account for repeated points
- i.e. $\beta_p = 1$ for a unique point (x_p, y_p)
- and $\beta_p = R$ and $\beta_p = 0$ for others if the point is repeated R times
- Repeated points should be evaluated (as for regression)
- Weights are fixed $\Rightarrow$ weighted classification costs can be minimized by means of local optimization
- Weights can also be used to assigned *confidence* values to points
- $0 \leq \beta_p \leq 1$, with $\beta_p = 0$ for untrustworthy points and $\beta_p = 1$ for trustworthy points

Weighted binary classification

- Weighted classification can be used also for unbalanced datasets
- Assign weights inversely proportional to number of members in each class
- $P = |\Omega_{+1}| + |\Omega_{-1}|$
- $\beta_{+1} \propto \frac{1}{|\Omega_{+1}|}$
  $\beta_{-1} \propto \frac{1}{|\Omega_{-1}|}$
- Example:

